

Objective of the Problem Statement

- Component communication
- Routing & nested routing
- Context API mastery
- Hook-based state management
- Clean UI composition
- Real-world thinking

Problem Statement 2: Smart Healthcare Appointment & Records Portal

Overview

Design and develop a **single-page React application** for a healthcare organization to manage **patients, doctors, appointments, medical records, and user roles**.

The application must use **React Functional Components with Hooks only**, implement **React Router**, and use **Bootstrap** for UI and layout.

Core Objectives

- Implement a multi-page React SPA
- Use React Router for navigation
- Use Hooks only (useState, useEffect, useContext, useParams, useNavigate)
- Share data between deeply nested components
- Implement protected and role-based routes
- Use Bootstrap components for forms, tables, modals, and layout

Modules & Features

1. Layout Components

Header

- Application title
- Logged-in user info
- Navigation:
 - Dashboard
 - Patients
 - Doctors
 - Appointments
 - Records

- Profile
- Logout

Footer

- Legal notice and version info
- Visible on all pages

2. Authentication & User Context

- Login with:
 - Username
 - Role (Admin / Doctor / Receptionist)
- Store login info in Context
- Protect all routes except login

3. Routing Structure

Route	Component
/login	Login
/dashboard	Dashboard
/patients	PatientList
/patients/:id	PatientDetails
/doctors	DoctorList
/appointments	AppointmentList
/records/:patientId	MedicalRecords
/profile	UserProfile

4. Dashboard

- Summary cards:
 - Total Patients
 - Active Doctors
 - Today's Appointments
- Data passed from parent state to reusable **InfoCard** components

5. Patient Management

PatientList

- List patients in a table
- Clicking a patient navigates to details page

PatientDetails

- Shows patient info
- Child components:
 - AppointmentHistory
 - AssignedDoctor
- Child components must send updates back to parent

6. Appointment Module

- Create new appointments
- Assign doctor to appointment
- Filter appointments by date or doctor

7. Medical Records

- Display patient medical records
- Accessible only to Admin and Doctor roles
- Uses route parameters and shared context

8. Role-Based Access Control

Role	Access
Admin	Full access
Doctor	View patients & records
Receptionist	Manage appointments only

Folder Structure (Suggested)

components/

pages/

context/

data/

App.jsx

Constraints

- ✓ Hooks only
- ✓ No backend

- ✓ Bootstrap only
- ✓ Clean component separation

Problem Statement 3: Online Learning Management System (LMS)

Overview

Build a **React-based Learning Management System** to manage **courses, instructors, students, enrollments, and progress tracking**.

The application must be built using **functional components and hooks only**, use **React Router**, and **Bootstrap** for styling.

Core Objectives

- Create a modular SPA with multiple pages
- Implement nested routing
- Manage global state using Context API
- Pass data across multiple component levels
- Implement protected routes based on user role

Modules & Features

1. Layout Components

Header

- Logo and app name
- Navigation:
 - Dashboard
 - Courses

- Instructors
- Students
- Enrollments
- Profile
- Logout

Footer

- Copyright
- Version number

2. Authentication & Context

- Login with:
 - Username
 - Role (Admin / Instructor / Student)
- Store authentication data in Context
- Protect all routes except login

3. Routing Structure

Route	Component
/login	Login
/dashboard	Dashboard
/courses	CourseList
/courses/:id	CourseDetails
/instructors	InstructorList
/students	StudentList
/enrollments	EnrollmentPage
/profile	UserProfile

4. Dashboard

- Summary cards:
 - Total Courses
 - Active Students

- Instructors
- Use reusable **SummaryCard** components

5. Course Management

CourseList

- Display all courses
- Clicking a course navigates to details

CourseDetails

- Shows:
 - Course description
 - Instructor
 - Enrolled students
- Child components:
 - StudentEnrollment
 - CourseProgress
- Data must flow both ways between parent and child

6. Enrollment & Progress Tracking

- Enroll students into courses
- Track completion percentage
- Progress updates must propagate across components

7. Role-Based Access Control

Role	Permissions
Admin	Full system access
Instructor	Manage assigned courses
Student	View enrolled courses only

Technical Constraints

- ✓ Hooks only
- ✓ React Router
- ✓ Bootstrap only
- ✓ No Redux
- ✓ No backend